

# Actividad N5: Datos Estructurados

Martina Araujo, Thiago Goya y Sofia Salaberry,

1

**Participantes:**

Martina Araujo, Thiago Goya y Sofia Salaberry

**Escuela:**

Escuela Técnica N°1 de Vicente Lopez

**Curso:**

Ciclo Superior 5°3°

**Docentes a cargo:**

Yamil Ganduglia, York Elias Mansilla Muñoz

**Materia:**

Lab. Programación

**Año:** 2026

**Repositorio de GitHub:**

Goya: [https://github.com/GoyaYuki/Datos Estructurados](https://github.com/GoyaYuki/Datos_Estructurados)

# Actividad N5: Datos Estructurados

Martina Araujo, Thiago Goya y Sofia Salaberry,

2

## Introducción

En esta actividad trabajamos con el modelado de datos mediante estructuras (struct) y punteros en C++. El objetivo principal fue aprender a agrupar diferentes tipos de datos dentro de una misma estructura, representando una entidad de manera más organizada. También aprendimos a utilizar punteros para acceder y modificar los datos de una estructura mediante el operador flecha (->).

Además, trabajamos con el pasaje de datos por referencia y con el manejo del buffer de entrada utilizando `cin.ignore()`. Estos conceptos nos permitieron entender un poco mejor cómo se almacenan y modifican los datos en la memoria mientras se ejecuta un programa.

## Objetivos

El objetivo de esta actividad fue aprender a crear y utilizar estructuras (struct) en C++ para agrupar diferentes datos relacionados dentro de una misma entidad. También buscamos comprender cómo funcionan los punteros, cómo pasar la dirección de una variable a una función y cómo utilizar el operador flecha (->) para acceder y modificar los datos de una estructura. Además, trabajamos con `cin.ignore()` para limpiar el buffer de entrada antes de utilizar `cin.getline()`.

## Desarrollo de la actividad

Durante la actividad creamos una estructura llamada `EntidadProyecto` para poder agrupar distintos datos relacionados dentro de una misma entidad. La estructura utilizada contiene un identificador, un nombre y una métrica:

```
struct EntidadProyecto {  
    int id;  
    char nombre[50];  
    float metrica;  
};
```

Después, en la función `main()`, se creó una variable de tipo `EntidadProyecto`. A partir de esta variable se pasó su dirección a una función mediante el operador `&`.

```
cargarDatos(&miEntidad);
```

La función recibe esa dirección mediante un puntero, lo que permite trabajar directamente sobre la estructura original.

```
void cargarDatos(EntidadProyecto* ptr);
```

Para acceder a los miembros de la estructura desde el puntero utilizamos el operador flecha (->), por ejemplo:

```
cout << "\n--- INGRESO DE DATOS MEDIANTE OPERADOR FLECHA ---" << endl;  
cout << "=> Ingrese el ID de la entidad (entero): ";  
cin >> ptr->id;  
cin.ignore();  
cout << "=> Ingrese el Nombre o Descripcion: ";  
cin.getline(ptr->nombre, 50);  
cout << "=> Ingrese la Metrica de Operacion (decimal/float): ";  
cin >> ptr->metrica;
```

# Actividad N5: Datos Estructurados

Martina Araujo, Thiago Goya y Sofia Salaberry,

3

También utilizamos `cin.ignore()` antes de `cin.getline()` para evitar problemas con el salto de línea que queda en el buffer después de ingresar datos mediante `cin`

De esta manera, pudimos ingresar correctamente un nombre completo, incluso cuando contiene espacios.

## CUESTIONARIO TÉCNICO DE CONTROL

1. **Heterogeneidad:** ¿Qué ventaja ofrece agrupar los datos de un sensor o dispositivo en un struct en lugar de declarar muchas variables sueltas?
2. **Operador Flecha:** ¿Por qué el compilador de C++ no permite usar la notación de punto (`ptr.id`) cuando la variable es de tipo puntero?
3. **Eficiencia en RAM:** ¿Por qué el pasaje por referencia mediante punteros optimiza la memoria procesada frente al pasaje por copia?
4. **Buffer de Entrada:** ¿Qué comportamiento erróneo ocurre en la terminal si se omite la instrucción `cin.ignore()` antes de un `getline()`?
5. **Aplicación al Proyecto Integrador:** ¿Qué entidad o componente hardware de su Proyecto Integrador Anual representaron con esta estructura y qué variables componen sus miembros?

### Resolución:

1. La ventaja de usar un struct es que permite agrupar diferentes tipos de datos relacionados dentro de una misma estructura. Por ejemplo, podemos guardar el nombre, el valor y el estado de un sensor en una sola variable, en lugar de tener muchas variables separadas.
2. El operador se utiliza cuando trabajamos directamente con una variable de tipo struct, mientras que un puntero guarda una dirección de memoria. Por eso, cuando tenemos un puntero a un struct, usamos `->` para acceder a sus miembros, como `ptr->id`.
3. El pasaje por referencia mediante punteros es más eficiente porque no crea una copia completa de los datos. En lugar de copiar toda la estructura en otra zona de memoria, se pasa su dirección y se trabaja directamente sobre los datos originales.
4. Si no usamos `cin.ignore()` antes de `getline()`, puede quedar un salto de línea pendiente en el buffer de entrada después de utilizar `cin`. Entonces, `getline()` puede leer ese salto inmediatamente y dejar el texto vacío, haciendo que parezca que el programa se saltó la entrada.
5. En nuestro Proyecto Integrador Anual de la Cooperadora Escolar, podemos representar una entidad como Producto mediante un struct. Sus miembros pueden ser, por ejemplo, el id del producto, nombre, precio, talle, color y cantidad disponible, permitiendo agrupar toda la información relacionada con cada producto en una sola estructura y facilitar su manejo dentro del sistema.

# Actividad N5: Datos Estructurados

Martina Araujo, Thiago Goya y Sofia Salaberry,

4

## Resultados obtenidos

Con esta actividad logramos poner en práctica el uso de estructuras y punteros en C++. Pudimos crear una entidad utilizando un struct, cargar sus datos mediante una función y acceder a ellos a través de un puntero utilizando el operador flecha.

También comprobamos la importancia de utilizar correctamente `cin.ignore()` antes de `cin.getline()`, ya que de esta manera se pueden ingresar datos de texto sin que el programa saltee la entrada.

Los conceptos trabajados en esta actividad pueden aplicarse al Proyecto Integrador Anual, ya que permiten organizar mejor la información de entidades como los productos y facilitan su manejo dentro del programa.

## División de tareas

| Participantes   | Tarea realizada                  |
|-----------------|----------------------------------|
| Sofia Salaberry | Informe y manual del programador |
| Martina Araujo  | Captura de evidencia y readme    |
| Thiago Goya     | Readme y License                 |

## Evidencias

Ejecución código

```
PS C:\Users\Goyitas\Desktop\Datos_Estructurados\src> cd "c:\Users\Goyitas\Desktop\Datos_Estructurados\src" ; if ($?) { g++ main.cpp -o main } ; if ($?) { .\main }
=====
MODELADO STRUCT - ESTUDIANTE: Goya Thiago
=====

--- INGRESO DE DATOS MEDIANTE OPERADOR FLECHA ---
=> Ingrese el ID de la entidad (entero): 25
=> Ingrese el Nombre o Descripcion: libro
=> Ingrese la Metrica de Operacion (decimal/float): 2.5

=== DATOS VERIFICADOS EN LA MEMORIA RAM ===
ID Registrado:      25
Nombre Registrado:  libro
Metrica Guardada:   2.5
Direccion RAM Hexadecimal: 0x2095dffb0
```

## Conclusión

En conclusión, esta actividad nos permitió comprender mejor cómo se pueden organizar los datos en C++ mediante el uso de estructuras (struct) y cómo se pueden modificar esos datos desde una función utilizando punteros.

También aprendimos a diferenciar el uso del operador `.` y del operador flecha (`->`), entendiendo que este último se utiliza cuando trabajamos con un puntero a una estructura. Además, aprendimos la función de `cin.ignore()` y su importancia al momento de utilizar `cin.getline()`.